Sisteme de operare

Laborator 11

Semestrul I 2023-2024

Vlad Olaru

Laborator 11

- advanced I/O
  - memory-mapped files
- programare client-server TCP/IP
  - demoni
  - server HTTP elementar

Memory-mapped files

- incarca continutul unui fisier intr-un buffer de memorie a.i. accesul la un element din buffer sa rezulte intr-un acces la disc
  - citirea unui element din buffer genereaza o citire din fisier
  - scrierea unui element din buffer rezulta intr-o scriere automata in fisier
  - astfel se executa I/O fara apelurile sistem *read*/*write*
- e nevoie de un apel sistem care sa instruiasca kernelul sa mapeze o regiune a unui fisier intr-o regiune din memoria RAM

caddr_t mmap(caddr_t addr, size_t len, int prot, int flag,

int fd, off_t off);

  - caddr_t e in general char *
  - intoarce adresa zonei de memorie in care e mapat fisierul

Argumente mmap

- *addr* = adresa de inceput e zonei de memorie unde vrem sa mapam continutul fisierului
  - in general se foloseste 0 pt a lasa kernelul sa aleaga adresa
  - daca *addr* != 0, trebuie sa fie multiplu al paginii de memorie virtuala obtinuta cu *sysconf(_SC_PAGESIZE)*
- *fd* = descriptorul fisierului al carui continut se mapeaza in memorie
  - fisierul trebuie deschis in prealabil
- *len* = nr de octeti mapati
- *off* = offsetul de start in fisier de unde se mapeaza octeti in memorie
  - multiplu al paginii de memorie virtuala
- *prot* = PROT_READ, PROT_WRITE, PROT_EXEC, PROT_NONE
  - trebuie sa se potriveasca cu modul in care a fost deschis fisierul
  - ex: *mmap(…,PROT_WRITE, …, fd,…)* nu merge cu *open(fd, O_RDONLY)*

Argumente mmap (cont.)

- *flag*
  - MAP_FIXED: valoarea de retur trebuie sa fie egala cu *addr*
    - daca nu e specificat si *addr* != 0, kernelul considera *addr* o sugestie
    - pt portatibilitate maxima *addr* = 0
  - MAP_SHARED: afecteaza operatiile de scriere in zona de memorie
    - daca flagul e setat => scrierea in buffer rezulta automat intr-o scriere pe disc
    - altfel spus, alte procese care partajeaza fisierul vad modificarea
  - MAP_PRIVATE: scrierile in zona mapata rezulta in crearea unei copii a fisierului mapat; toate referintele ulterioare la elementele modificate sunt efectuate asupra copiei
  - orice apel *mmap* **trebuie** sa foloseasca fie MAP_SHARED fie MAP_PRIVATE
  - MAP_ANONYMOUS: zona de memorie nu are asociata un fisier
    - initializata cu 0
    - *fd* si *off* sunt ignorate
    - impreuna cu MAP_SHARED implementeaza o zona de memorie partajata intre procese inrudite

Observatii

- Q: pt off ! = 0, ce se intampla daca dimensiunea zonei de memorie mapate nu e multiplu al paginii sistemului?
  - ex: pp. dimensiunea fisierului e 96 octeti si pagina sistemului 4096

  => dupa *mmap* se obtine o regiune de memorie de 4096 de octeti, dintre care ultimii 4000 sunt setati pe 0

  Obs: daca se scrie dincolo de offsetul 96, modificarile nu se reflecta in fisier

- semnale
  - SIGSEGV – generat daca pt acces la o zona de memorie la care nu avem acces
    - generat si daca scriem intr-o zona de memorie RO
  - SIGBUS – generat de accese fara sens intr-o portiune a zonei de memorie mapate
    - ex: fisier mapat de un proces in intregime
      - alt proces trunchiaza fisierul inainte ca primul proces sa acceseze fisierul mapat
      - accesul la zona de memorie care a fost trunchiata rezulta in SIGBUS

- o regiune de memorie mapata pentru un fisier se mosteneste de catre copil dupa *fork*, evident nu si daca se apeleaza ulterior *exec* !

munmap

- zona de memorie se demapeaza:
  - implicit, la sfarsitul procesului (prin *exit*)
  - explicit cu *munmap*
  - inchiderea descriptorului de fisier nu demapeaza regiunea de memorie

int munmap(caddr_t addr, size_t len);

- nu afecteaza fisierul mapat (continutul ei nu e trimis pe disc)
- cu MAP_SHARED modificarile sunt automat scrise pe disc, nu cu *unmap* !

Concluzii mmap

- metoda mai rapida la copierea fisierelor (vizibil in cazul fisierelor mari)
- limitari: nu putem folosi *mmap* pentru conexiuni de retea sau pentru device-uri speciale (eg, terminal)
- e nevoie de atentie daca dimensiunea fisierului mapat se schimba dupa mapare
- in general simplifica algoritmii ori de cate ori manipularea unui buffer de memorie e preferabila operatiilor de I/O explicite (*read*/*write*)

Demoni

- procese care ruleaza in background si nu sunt asociate cu un terminal de control
- in general porniti de scripturile sistemului de operare la sfarsitul operatiei de boot
- ce se intampla insa cand un server e pornit de utilizator dintr-un shell (terminal)?
  - trebuie sa se dezasocieze de terminal pentru a evita interactiuni nedorite cu controlul proceselor executat de shell, gestiunea sesiunii de terminal sau pt a nu tipari mesajele server pe terminal (avand in vedere ca ruleaza in background)
- exemple demoni
  - serviciile sistem din */etc/rc* pornite cu privilegii de superuser (*inetd/xinetd*, *Web*, *sendmail/postfix*, *syslogd* etc)
  - job-uri *cron/at* invocate de demonul de *cron*
  - servere/programe pornite din terminal

Output pt demoni

- pentru ca demonii nu au terminal de control au nevoie de metode specifice pt a afisa mesajele atunci cand au nevoie (ex mesaje de eroare, urgenta, logging, etc)
- metoda standard foloseste functia *syslog* care trimite mesaje catre demonul *syslogd (rsyslogd* pe sisteme Linux*)*
  - in general exista un fisier de configurare *syslog.conf* care specifica ce trebuie facut cu fiecare mesaj primit de demon:
    - se adauga la sfarsitul unui fisier
    - se logheaza pe consola *(/dev/console*)
    - se forwardeaza catre demonul *syslogd* al altei masini
  - serviciul functioneaza pe portul 514 UDP (v. */etc/services*)
  - la primirea semnalului SIGHUP se reciteste fisierul de configurare (in sistemele moderne se foloseste comanda *service reload*)
    - ex: $ kill –HUP <pid syslogd>
- mesajele se pot trimite direct pe portul 514 UDP, dar uzual se foloseste functia *syslog*

Functia syslog

- in lipsa terminalului de control demonul nu poate folosi functii de genul *fprintf* la *stderr*
- logarea mesajelor se face cu functia *syslog*

  #include <syslog.h>

  *void syslog(int priority, const char *message, …);*
- primul argument este o combinatie intre *nivel* si *facilitate* (*level/facility*)
- mesajul (al doilea parametru) este un string de formatare ca la *printf*
  - *%m* poate fi folosit pt a loga mesajul de eroare corespunzator valorii curente a *errno*
- mesajele logate au un nivel intre 0 si 7
- daca utilizatorul nu specifica nici un nivel se foloseste implicit LOG_NOTICE (nivel 5)
- nivelul permite ca toate mesajele de un anumit nivel sa fie tratate la fel

Functia syslog (cont.)

- mesajele logate contin o *facilitate* care identifica procesul care trimite mesajul
- valoarea implicita pentru *facilitate* este LOG_USER
- ex: mesaj de eroare demon care nu reuseste sa redenumeasca un fisier

  syslog(LOG_INFO|LOG_LOCAL2, "rename (%s,%s): %m", file1, file2);

- valoarea facilitatatii permite ca toate mesajele venite de la o anumita facilitate sa fie tratate la fel in syslog.conf
  - ex:

```
kern.*          /dev/console
local7.debug    /var/log/cisco.log
```

Functia syslog (cont.)

- la primul apel *syslog* se creeaza un socket Unix UDP care se conecteaza la portul 514 si care ramane conectat pana cand se termina procesul
- alternativ, se pot folosi functiile *openlog/closelog*

```
#include <syslog.h>

void openlog(const char *ident, int options, int facility);

void closelog((void);
```

- *ident* este un string care va prefixa toate mesajele logate prin *syslog*
- *options* este o valoare de OR logic intre mai multe constante (v. man page)
- daca se specifica *facilitatea* la *openlog*, ulterior cand se apeleaza *syslog* nu se mai paseaza ca prim parametru decat *nivelul*
- mesajele se pot loga si cu comanda *logger* (din linie de comanda)

Structura demoni

- (1) detasarea de terminal
  - se apeleaza *fork* si procesul parinte se termina imediat si copilul continua
  - cand parintele termina, shell-ul crede ca s-a terminat comanda, ca atare copilul ruleaza automat in background
  - copilul mosteneste GID-ul parinte dar are propriul PID, fapt ce face ca el sa nu poata fi liderul grupului de procese
  - copilul apeleaza *setsid* pentru a crea o noua sesiune si a deveni lider de grup in noua sesiune
    - copilul nu mai are terminal de control in acest moment
  - copilul se protejeaza de semnalul SIGHUP si apeleaza *fork* din nou
  - primul copil, acum parinte, termina si copilul sau (al doilea copil) continua sa ruleze
  - al doilea apel *fork* garanteaza ca demonul nu poate capata un terminal de control daca decide in viitor sa deschida un terminal
    - Cand un lider de sesiune fara terminal deschide un terminal, acel terminal devine terminalul de control al liderului sesiunii
    - al doilea apel fork garanteaza ca al doilea copil nu mai e lider de sesiune si nu mai poate obtine un terminal de control
  - SIGHUP trebuie ignorat pt ca atunci cand liderul de sesiune (primul copil) se termina, toate procesele din sesiune (al doilea copil) primesc semnalul SIGHUP

Structura demoni (cont.)

- (2) schimbarea directorului de lucru curent
  - demonul schimba directorul de lucru curent in / (sau alt director la alegere)
  - daca demonul genereaza un fisier core, acesata se salveaza in directorul de lucru current
  - daca directorul de lucru al demonului era pe un sistem de fisiere si demonul continua sa lucreze acolo, sistemul de fisiere respectiv nu va putea fi demontat
- (3) inchiderea tuturor descriptorilor de fisiere deschisi
  - se inchid toti descriptorii de fisiere mosteniti de la procesul care a creat demonul (in mod normal, shell-ul)
- (4) redirectarea stdin, stdout si stderr la /dev/null
  - garanteaza ca descriptorii de fisiere corespunzatori sunt deschisi, dar citirea intoarce EOF iar scrierea se pierde
  - necesar pt ca functiile de biblioteca care presupun folosirea acestor descriptori sa nu esueze
- (5) utilizarea syslogd pentru logarea erorilor
  - apel *openlog* in mod normal cu numele procesului (argv[0])